MACs & Authenticated Encryption

Computer Systems Security
2025/26
LEI - UM

Hugo Pacheco hpacheco@di.uminho.pt

Hash Functions

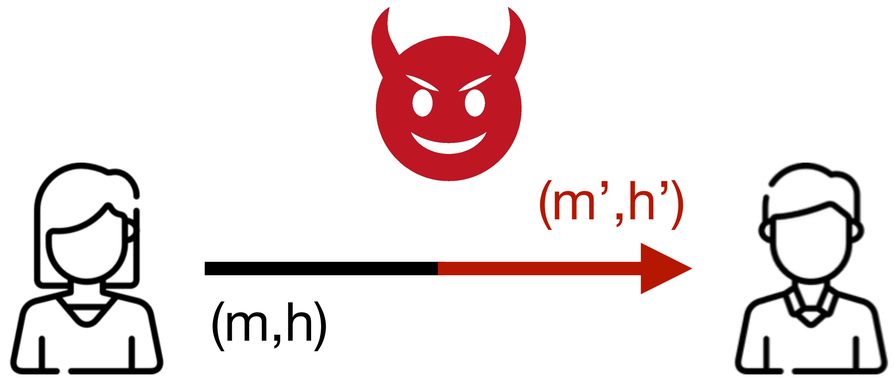
A colloquial use case

- Alice wants to download a file
- Attacker performs a man-in-the-middle or a supply-chain attack
  - He could change the file (find a 2nd pre-image) for the same hash
    - Nearly impossible, even for the still common MD5
  - He could create a new file/hash pair
    - Trivial if he controls the website / channel
- Alone, **hash functions do not ensure integrity nor authenticity of data!**
  - This can be solved with digital signatures (asymmetric crypto)

Hash Functions

Another use case

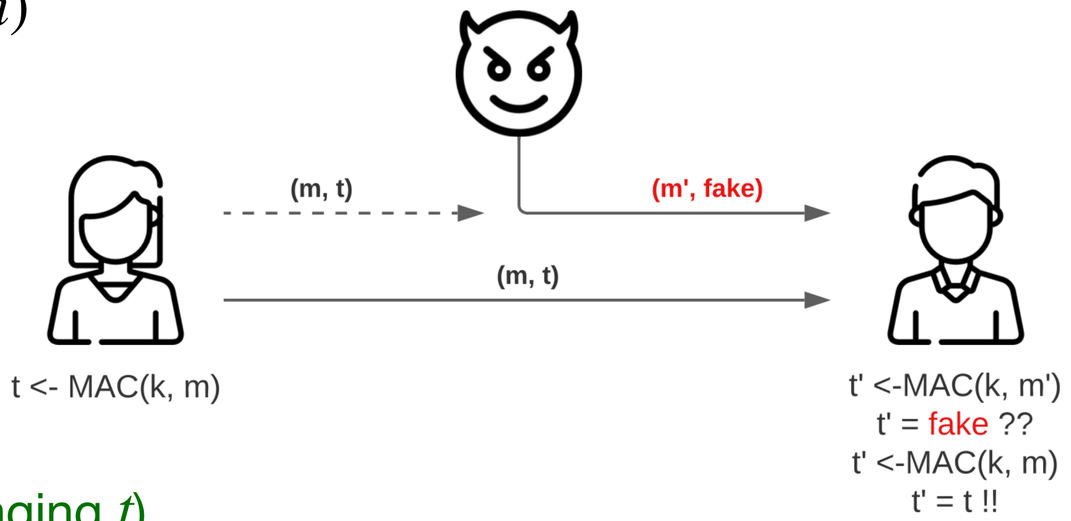
- Alice wants to send a public message to Bob
- Attacker performs a man-in-the-middle attack
  - He could create a new message/hash pair
    - Trivial if he controls the channel
- Alone, **hash functions do not ensure integrity nor authenticity of data!**
  - This can be solved with MACs (symmetric crypto)



- Why symmetric?
  - Alice and Bob will need to hold a shared key

Message Authentication Codes (MACs)

- Alice wants to send a public message to Bob
  - Assumed some setup/agreement to establish a secret key known only to legitimate parties
  - Alice computes an **authentication tag** $t \leftarrow \text{MAC}(k, m)$
  - Alice sends (m, t)
  - Bob recomputes $t' \leftarrow \text{MAC}(k, m)$
  - Bob compares tags, rejects the message if $t \neq t'$
- **MACs ensure integrity and authenticity of data!**
  - Attacker cannot change the message m (without changing t)
  - Attacker cannot compute t (without knowing k)



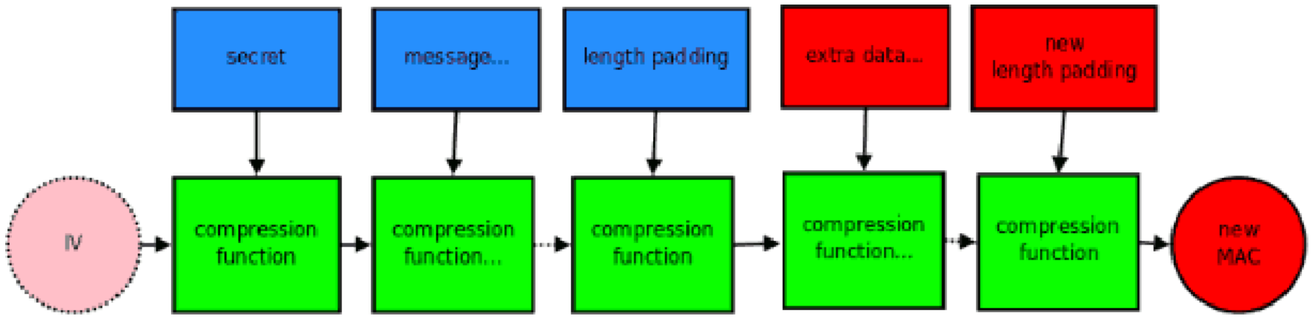
MACs

Keyed hash functions

- A MAC can be thought of as a **keyed hash function**
  - Hash function with a secret key plus a message
  - The input to the hash function is now **not completely public**
    - Attacker can know the public message m
    - Attacker shall not know the secret key k
- Simplest construction: $\mathrm{MAC}(k, m) = \mathrm{H}(k \parallel m)$
- Can anything go wrong?
  - Can the attacker forge a new message/MAC pair without knowing the secret key?

Keyed Hash Functions
Length extension

- For MAC(k, m) = H$(k \parallel m)$, consider this scenario
  - Assume an attacker knows m and H$(k \parallel m)$
  - Can he calculate H$(k \parallel m \parallel pad \parallel m')$?
    - Easy to determine the original padding pad
    - Attacker knows the size of $k \parallel m$
    - Does he need to know the secret key k to compute MAC$(k, m \parallel pad \parallel m')$?

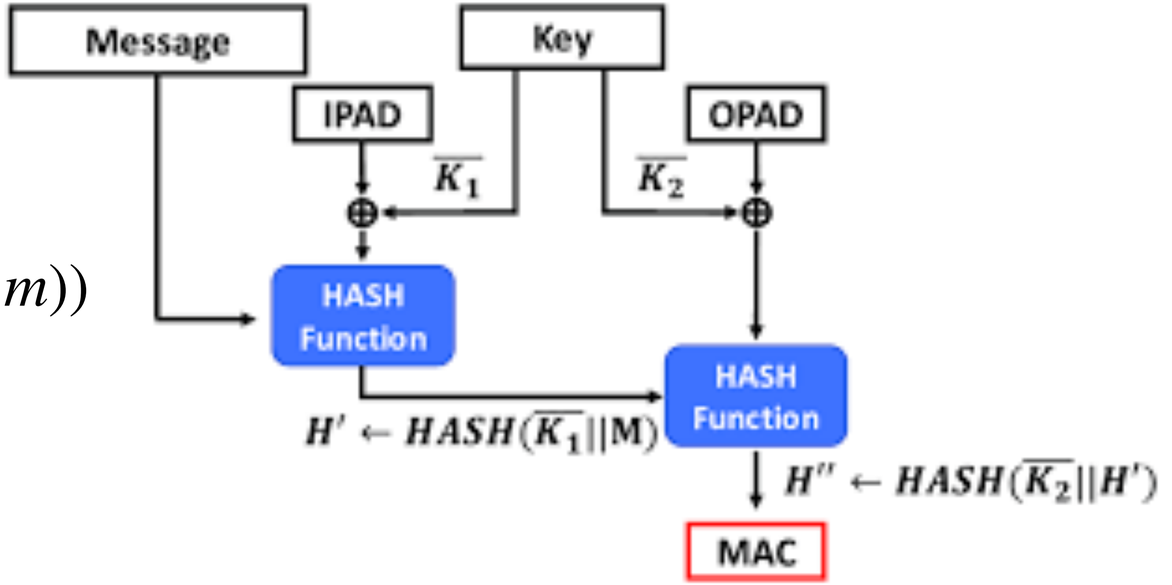


- **This MAC construction is vulnerable to length extension attacks**
  - E.g., the output of the MD construction (SHA-2 and before) is the internal state after processing the last block
    - Attacker can compute t' = H$(k \parallel m \parallel pad \parallel m')$ just from t = H$(k \parallel m)$ and m'

Keyed Hash Functions

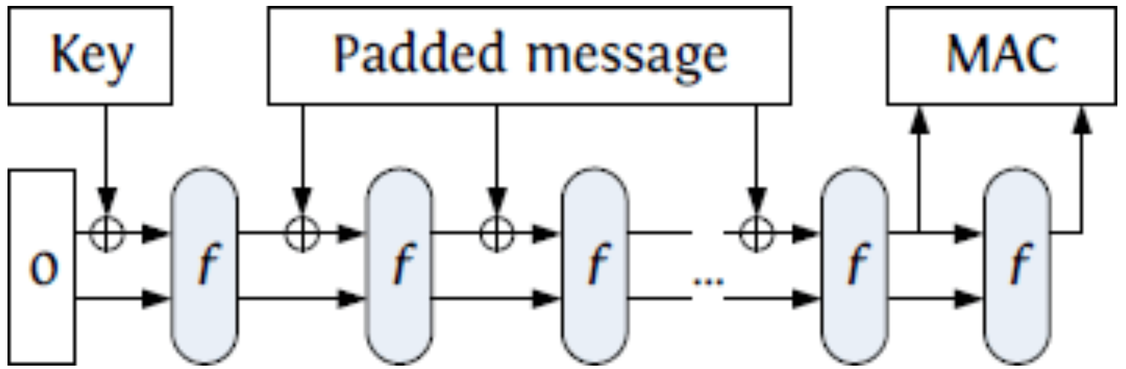
HMAC construction

- A standard construction is the Hash-based Message Authentication Code (HMAC)
- To avoid length extension attacks
- It performs two passes on the hash function
  - $\text{HMAC}(k, m) = \text{H}((k \oplus opad) \parallel \text{H}((k \oplus ipad) \parallel m))$
  - $ipad$ and $opad$ are fixed constants
    - That align to block size
    - Chosen to have high “Hamming distance”
- Examples: HMAC-SHA256, etc



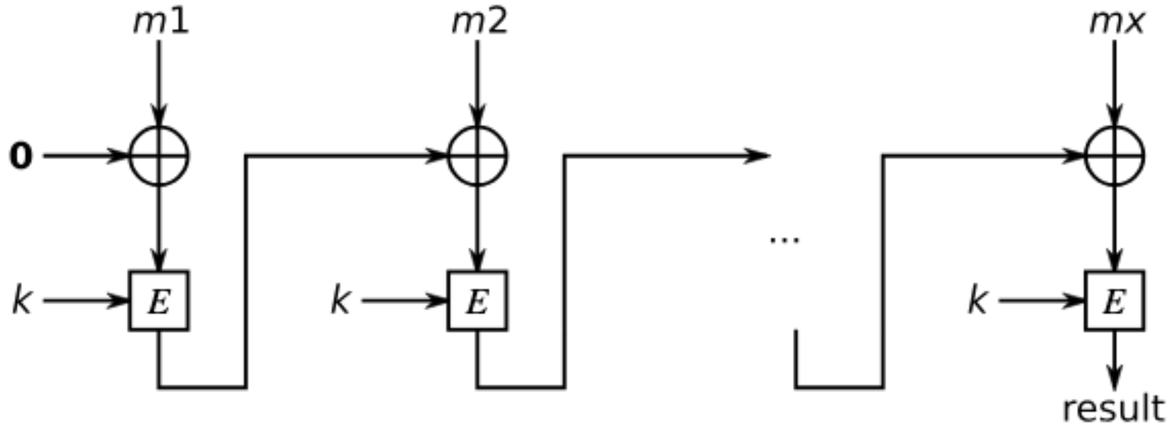
Keyed Hash Functions
Length extension

- Another main advantage of the sponge construction (SHA-3)
  - **It is immune to length extension attacks**
    - The capacity *c* is never seen by the user
      - Impossible to predict the internal state necessary to extend the hash
  - We don’t need to use the HMAC construction!
    - The simple construction is secure
      - $\mathrm{MAC}(k, m) = \mathrm{H}(k \parallel m)$



MACs
From block ciphers

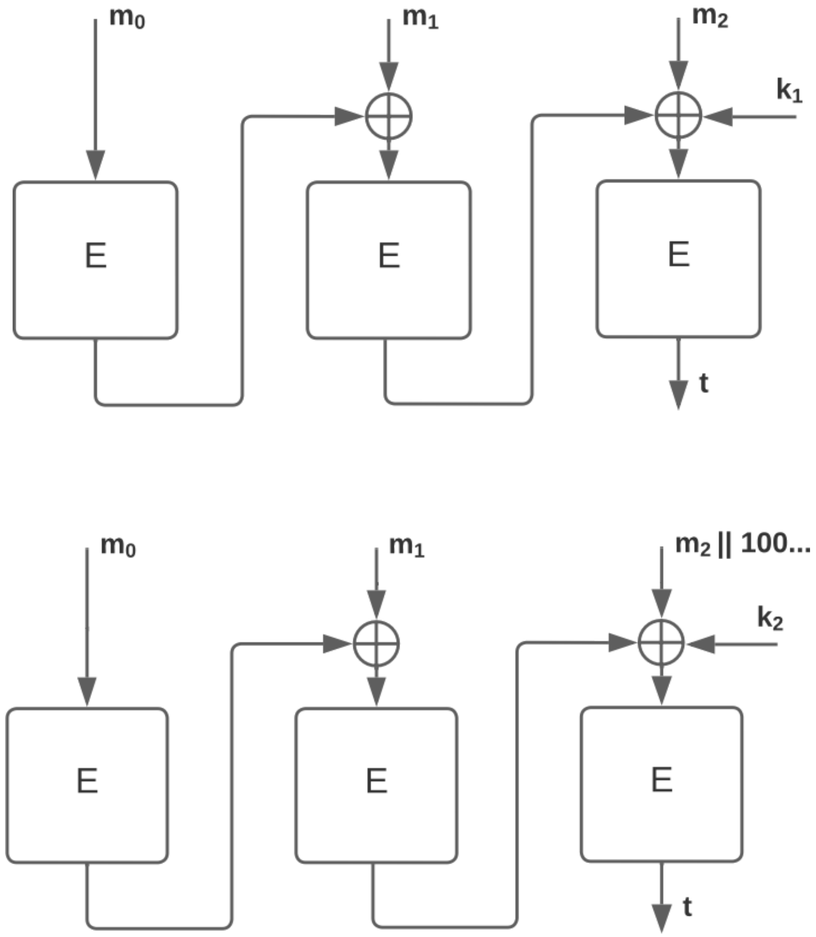
- Roadmap so far: block ciphers ⇒ hash functions ⇒ MACs
  - E.g., SHA-2 uses the Davies-Meyer (DM) construction to build its compression function from the block cipher SHACAL
- There are also direct constructions: block ciphers ⇒ MACs
  - CBC-MAC
    - Use the CBC mode of a block cipher
    - Define IV = 0
    - Authentication tag = ciphertext of last block
    - **Vulnerable to length extension attacks!**



MACs From Block Ciphers

CMAC

- **Fixes CBC-MAC by processing last block differently**
  - Everything else like CBC-MAC
  - Derive two keys k_1 and k_2 from k
  - If the message is a perfect multiple of the block size: last block is XORed with k_1 before the final encryption (top)
  - If the message needs padding: last block is padded and then XORed with k_2 before the final encryption (down)



MACs
From UHFs

- Universal Hash Functions (UHFs) are a weaker form of hash functions
  - Parametrised by a key: UH(k, m)
  - ϵ**-Universal** (for two messages $m_0 \neq m_1$ and random k)
    - $Pr[\text{UH}(k, m) = \text{UH}(k, m')] \leq \epsilon$
    - Collision probability bounded by a very small ϵ
    - Much weaker than collision resistance: UH$(\$, m)$ vs H(m)
- A UHF can be directly used as a MAC
  - **Provided that we only authenticate one message!**

MACs From UHFs

Wegman-Carter construction

- **We can use a PRF to strengthen a UHF into a fully secure MAC**
  - Essentially “encrypting” the universal hash value
- We can fill the PRF role with:
  - A block cipher (e.g., AES) for computational security
  - OTP for information-theoretic security

- Construction:

$$k = (k_1, k_2)$$

$$\text{MAC}(k, m) = \text{UH}(k_1, m) \oplus \text{PRF}(k_2, n)$$

n is a public Nonce

MACs From UHFs

ChaCha20-Poly1305

- A widely used UHF is Poly1305
  - Message chunks $m_1, \ldots, m_n$ are the coefficients of a polynomial modulo a fixed prime $2^{130} - 5$

$$\text{Poly1305}((k_1, k_2), m) = (m_1 k + \ldots + m_n k^n \pmod p) + PRF(k_2, n)$$

- Most frequently combined with ChaCha20
- Much faster than HMAC or CMAC in software
  - Does not rely on dedicated hardware support for hash functions or block ciphers
- Much simpler design

MACs + Encryption

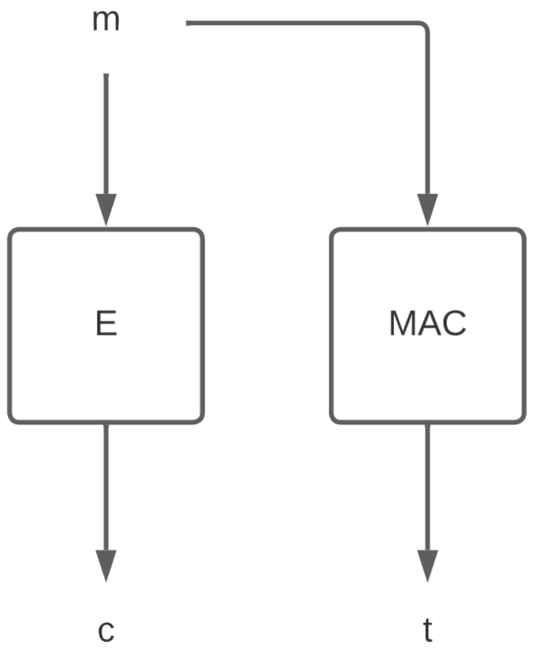
Authenticated Encryption

- In practice, any secure channel uses **Authenticated Encryption (AE)**
  - Messages need to be confidential
  - Messages need to be authentic
  - Messages shall not be repeated / omitted / removed
  - Encryption provides confidentiality; MACs provide authenticity
- Usually generalized to **Authenticated Encryption with Associated Data (AEAD)**
  - “Associated Data” = public meta-information (e.g. sequence numbers) which is not encrypted but is attached to the integrity guarantee

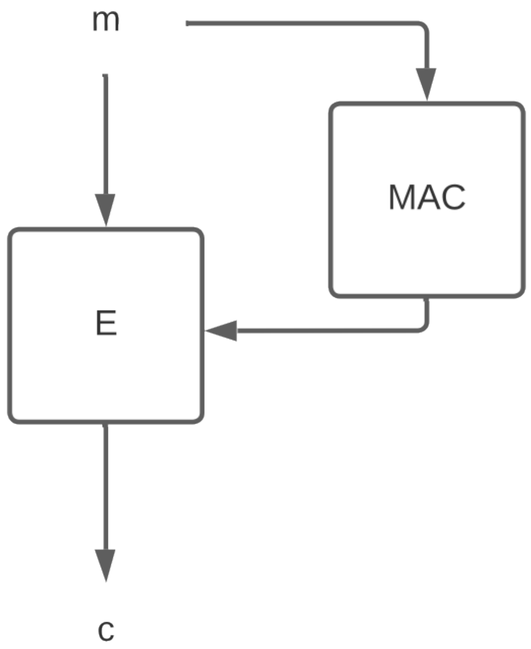
Authenticated Encryption

The good, the bad and the ugly

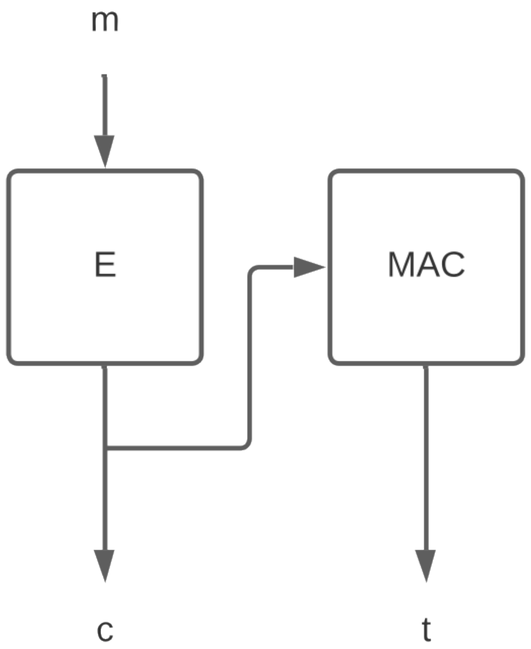
- **Encrypt-and-MAC:** encryption and MAC of the message



- **MAC-then-Encrypt:** encrypt message and its authentication



- **Encrypt-then-MAC:** authenticate the message encryption



Authenticated Encryption

Encrypt-and-MAC (the bad)

- Construction:

$$k = (k_1, k_2)$$
$$c \leftarrow_{\$} \mathsf{E}(k_1, m)$$
$$t \leftarrow \mathsf{MAC}(k_2, m)$$
Output: (c, t)

- Used in legacy SSH
  - Mostly secure, fell victim to side-channel attacks

- ✚ Parallel processing
- ➖ Potentially malicious c decrypted before authentication
- ➖ MACs are (typically) not designed to ensure confidentiality
- ➖ Vulnerable to side-channel attacks (failed decryption vs failed MAC)

Authenticated Encryption

MAC-then-Encrypt (the ugly)

- Construction:

$$k = (k_1, k_2)$$
$$t \leftarrow \mathsf{MAC}(k_1, m)$$
$$c \leftarrow_{\$} \mathsf{E}(k_2, m \parallel t)$$
Output: c

- Used in legacy TLS
  - Painful story with padding oracle attacks, e.g. Lucky 13

- ✚ Confidentiality of the MAC (actually mostly useless)
- ➖ Sequential processing
- ➖ Potentially malicious c decrypted before authentication
- ➖ Vulnerable to side-channel attacks (failed decryption due to padding)

Authenticated Encryption

Encrypt-then-MAC (the good)

- Construction:

$k = (k_1, k_2)$
$c \leftarrow_{\$} \mathrm{E}(k_1, m)$
$t \leftarrow \mathrm{MAC}(k_2, c)$
Output: (c, t)

- Used in IPSec
  - Useful against DoS attacks

- ✚ Secure against all “oracle” attacks
- ✚ Can reject faster when MAC fails
- ✚ Potentially malicious c not decrypted unless it is authenticated
- ➖ Sequential processing

Authenticated Encryption

Authenticated Encryption with Associated Data (AEAD)

- Modern trend is to actually move to AEAD “from scratch”
  - Encryption and the MAC are “intertwined” into a single operation
  - A specialized version of Encrypt-then-MAC under the hood
  - **Optimized for performance**
    - Parallel block processing
    - Higher throughput
    - Lower memory requirements

AEAD

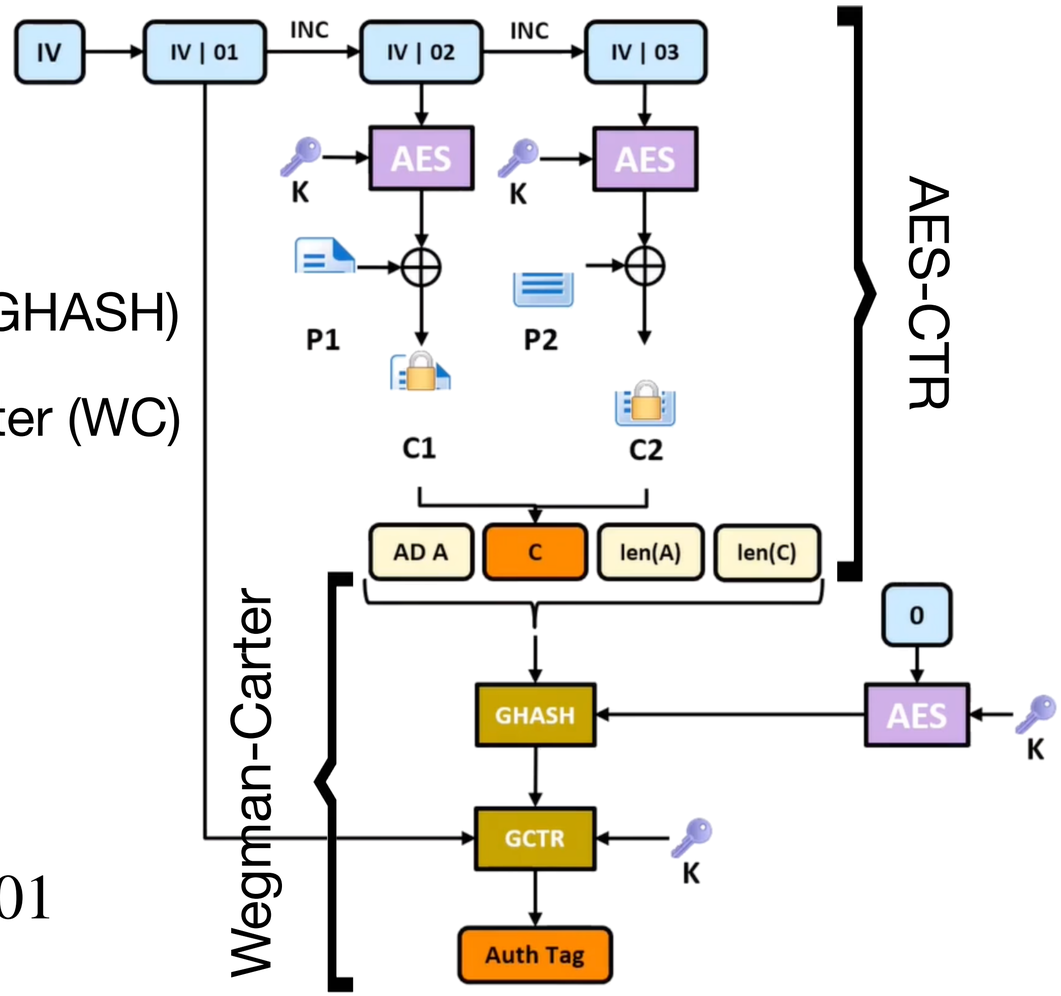
Galois/Counter Mode (GCM)

- The mostly used AEAD (IPSec, SSH, TLS, etc)
  - Block cipher in CTR mode + UHF
- The most prominent instantiation is AES-GCM (AES + GHASH)
- Remember Encrypt-then-MAC (EtM) and Wegman-Carter (WC)

$$\text{EtM}((k_1, k_2), m) = \text{E}(k_1, m) \parallel \text{MAC}(k_2, \text{E}(k_1, m))$$
$$\text{WC}((k_1, k_2), m) = \text{UH}(k_1, m) \oplus \text{PRF}(k_2, n)$$

- GCM effectively fuses EtM with WC

$$\text{GCM}(k, a, m) = \text{UH}(H, a \parallel \text{E-CTR}(k, m)) \oplus Mask$$
with $H = \text{E}(k,0)$ and $Mask = \text{E}(k, n)$ and $n = IV \parallel 01$



Galois/Counter Mode (GCM)

Performance

- ✚ Encryption layer inherits parallelism from CTR mode
- ➖ Authentication layer blocks if associated data a is known only in the end
- But if a is known from the start (public info, not an unreasonable scenario)
  - ✚ Ciphertext blocks computed and authenticated on the fly
  - ✚ Authentication of previous block is accumulated while current block is being encrypted

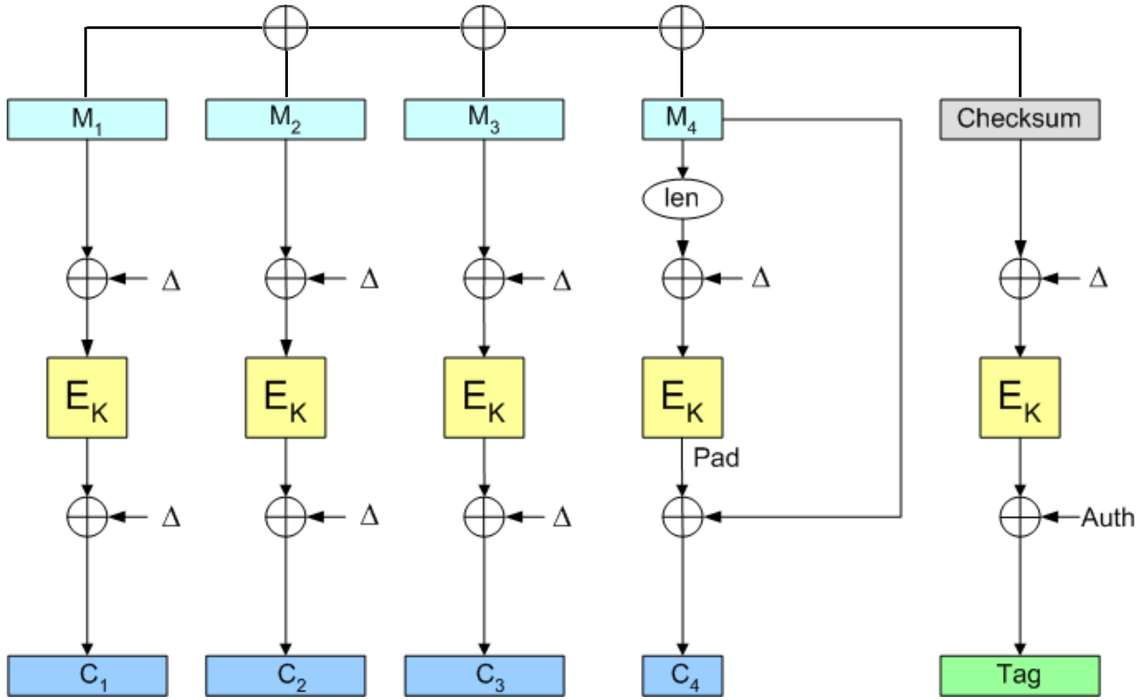
(ChaCha20-Poly1305 not covered, but has similar structure)

AEAD

Offset CodeBook (OCB)

- No UHF, block cipher does simultaneously encryption + authentication
- Applies a unique offset ($\Delta \leftarrow_{\$} \mathrm{E}(k, n)$) to each plaintext block
  - Before encryption (masking the plaintext input)
  - After encryption (masking the cipher's output)

$$\mathrm{OCB}(k, m_i) = \mathrm{E}(k, m_i \oplus \Delta_i) \oplus \Delta_i$$



Offset CodeBook (OCB)

Performance

- + Completely parallel, the perfectly streamable AEAD
  - + A single pass ($n + 1$ AES parallel calls, for n plaintext blocks)
- + Up to 3x faster than GCM in software
- − Complicated legal history (initially patented by Rogaway) hindered adoption
- − Unlike GCM (that uses CTR mode), requires both block cipher encryption and decryption
  - − Strong impact on hardware deployments

AEAD

Nonce reuse

- **Nonce reuse is deadly for both GCM and OCB**

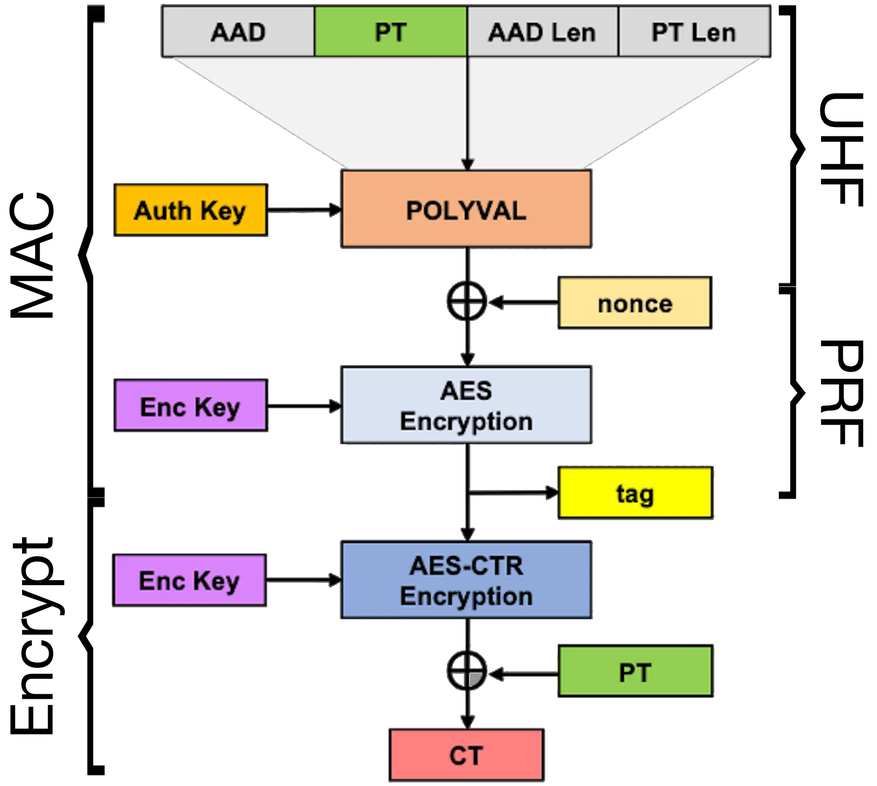
	Confidentiality	Integrity
GCM	Leaks XOR of multiple messages	Reveals hash key
OCB	Leaks block patterns	Block forgery via XOR sums

- Is this an actual issue?
  - It is surprisingly hard to enforce “Nonce uniqueness” in practice

Galois/Counter Mode (GCM)

Synthetic IV mode (SIV)

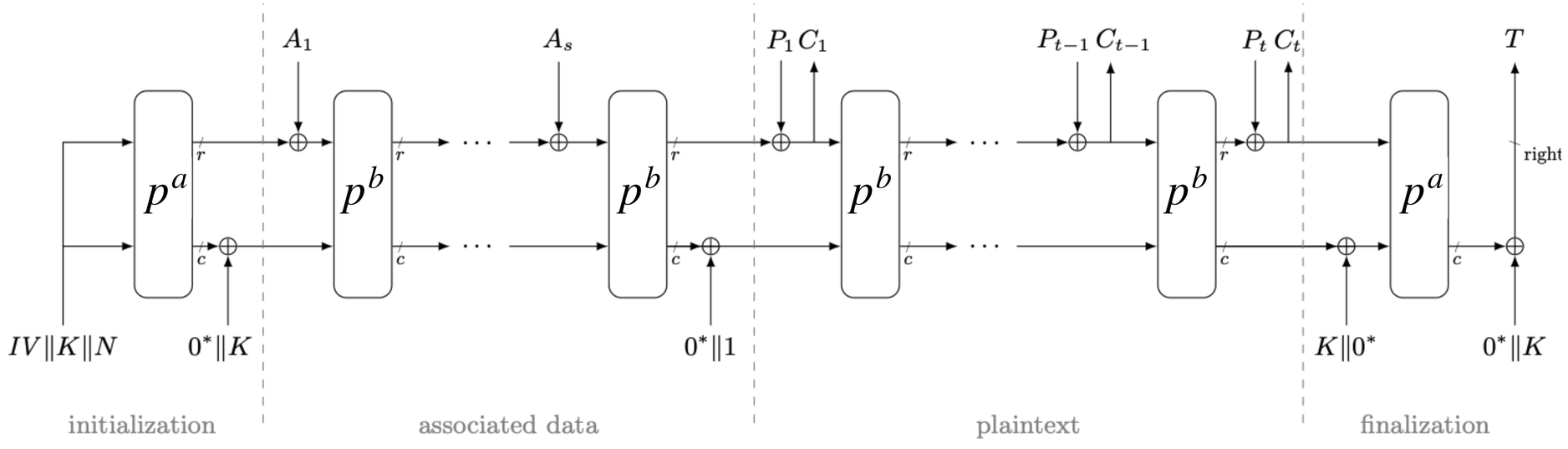
- A general construction that can be applied to any AEAD to make it **nonce-misuse resistant**
  - Effectively a variant of MAC-then-Encrypt
  - The Nonce / Synthetic IV is computed from the authentication tag
- AES-GCM-SIV is the standard (uses POLYVAL as a UHF)
- No longer streamable, requires two sequential passes over the plaintext



AEAD

Duplex sponge construction

- The sponge construction (SHA-3) can be turned into an AEAD cipher
- Ascon is the new standard (2023) for lightweight cryptography



Absorb key and nonce	Absorb the associated data	Squeeze bits to XOR with the plaintext + absorb the cyphertext	Squeeze the auth tag

Duplex Sponge Construction

Performance

- **Nonce-Misuse Resilient**
  - − Nonce reuse breaks confidentiality (in the same way as GCM)
  - + Nonce reuse does not break authenticity
- + Fast and streamable (a single pass over the inputs)
- + Encryption and authentication use the same permutation
- + Decryption is the same as encryption

(SIV construction could be applied, but not done in practice ⇒ lightweight)